

VDRSD

Virtual Distributed Replicated Storage Device

Project type	Individual Capstone Project
Track / domain	Wipro Embedded Track: Linux Device Drivers, System Programming & C++
Version	1.0
Date	October 2026

Table of Contents

1 Introduction	4
1.1 Project Overview	4
1.2 Problem Statement	4
1.3 Objectives	4
1.4 Expected Outcome	5
1.5 Application Areas	5
1.6 Key Project Specifications	5
2 System Architecture	6
2.1 High-Level Architecture	6
2.2 System Components	7
2.3 Data Flow	7
2.4 Architectural Pipeline	7
2.5 Fault-Tolerance and Recovery Flow	8
3 Requirements	9
3.1 Functional Requirements	9
3.2 Non-Functional Requirements	9
3.3 Development Timeline	10
4 System Design	11
4.1 Core Data Structures	11
4.2 Component Responsibilities	12
4.3 Design Patterns	12
4.4 UML Class Diagram	13
4.5 Sequence Diagram	14
4.6 State Machine	15
5 Kernel Module: vdrsd.c	16
5.1 Overview	16
5.2 Module Metadata	16
5.3 Character Device Interface	16
5.4 Ring Buffer	17
5.5 IOCTL Interface	17
5.6 Statistics Structure	17
5.7 Important Kernel Functions	18
5.8 Kernel Version Compatibility	19
6 User-Space Daemon: vdrsd_daemon.cpp	20
6.1 Overview	20
6.2 Thread Architecture	20
6.3 Kernel Interface	21
6.4 Storage Engine	21
6.5 Network Manager	21
6.6 Health Monitor	22
6.7 Ring Buffer	22
6.8 CRC32 Integrity	23
6.9 Cluster Synchronization	24
6.10 Graceful Shutdown	24

7 Build System and Project Structure	25
7.1 Required Tools	25
7.2 Installation	25
7.3 Root Makefile	26
7.4 Kernel Makefile	26
7.5 Project Directory Structure	27
7.6 Build Commands	27
8 Configuration and Cluster Setup	28
8.1 Configuration File Format	28
8.2 Cluster Topology	28
8.3 Node Configuration	28
8.4 Startup Script	29
8.5 Shutdown Script	29
9 Testing	30
9.1 Testing Strategy	30
9.2 Unit Testing	30
9.3 Integration Testing	30
9.4 Stress Testing	31
9.5 Memory Testing	31
9.6 Test Cases and Results	31
10 Execution and Demonstration	32
10.1 Build and Load Kernel Module	32
10.2 Start Daemon Cluster	33
10.3 Normal Data Write	34
10.4 Data Replication	34
10.5 Node Failure	35
10.6 Node Recovery and Synchronization	36
10.7 Important Terminal Outputs	37
11 Results and Discussion	38
11.1 Achievements	38
11.2 Fault-Tolerance Behavior	38
11.3 Data Integrity	38
11.4 Performance and Testing Results	39
12 Limitations and Future Work	40
12.1 Current Limitations	40
12.2 Future Improvements	40
13 Module Mapping and Assessment	41
13.1 Training Module Coverage	41
13.2 Assessment Evidence	41
13.3 Git Branching Strategy	42
13.4 Development Timeline	42
14 Conclusion	44
15 References and Project Resources	45

1 Introduction

1.1 Project Overview

VDRSD (Virtual Distributed Replicated Storage Device) is a fault-tolerant, distributed storage system. It integrates a Linux character device driver, a multi-threaded C++17 replication daemon, and TCP-based peer networking. The project was built from the ground up as an end-to-end Linux systems programming capstone.

User applications write data to the virtual character device `/dev/vdrsd` like a regular file. The kernel module buffers the data, the user-space daemon reads it, persists it to local disk, and replicates it to peer nodes over TCP.

1.2 Problem Statement

Modern distributed systems require fault-tolerant storage that can survive individual node failures without data loss. Building such a system demands deep integration across three disciplines simultaneously:

- **Linux kernel development:** character device drivers, ring buffers, spinlocks, wait queues.
- **System-level C++ programming:** POSIX I/O, daemon processes, signal handling, multithreaded design.
- **Network communication:** TCP socket programming, replication protocols, peer discovery.

There was a need for a single, cohesive capstone project that exercises all of these skills in an end-to-end pipeline, from raw kernel data capture to distributed network replication and persistent local storage.

1.3 Objectives

- Provide a virtual character device, `/dev/vdrsd`, that accepts data from user applications.
- Buffer written data in a kernel ring buffer and signal the user-space daemon.
- Implement a multi-threaded C++17 daemon that persists each 4 KB chunk to disk and replicates it, with CRC32 integrity verification, to all peer nodes over TCP.
- Continue operating in degraded mode when a peer node goes offline, and recover automatically through catch-up synchronization when it returns.
- Apply established design patterns (Singleton, Observer, Producer-Consumer, RAII) and SOLID principles.
- Demonstrate a professional development process from requirements through design, implementation, testing and final delivery.

1.4 Expected Outcome

Table 1. Expected outcomes of the VDRSD system

Outcome	Description
Write to <code>/dev/vdrsd</code>	User applications write data to the virtual character device just like any regular file.
Kernel capture	The kernel module buffers the data in a 256 KB ring buffer and wakes the user-space daemon.
TCP replication	The C++ daemon replicates each 4 KB chunk, with CRC32 integrity, to all peer nodes over TCP.
Persistent storage	Data is flushed to disk as <code>chunk_XXXXXXXX.dat</code> files using POSIX <code>write</code> and <code>fsync</code> .
Read from any replica	Data can be retrieved from any live replica node, providing redundancy and fault tolerance.
Fault tolerance	The system continues operating in degraded mode when a peer node goes offline.

1.5 Application Areas

- Educational demonstration of end-to-end Linux systems programming.
- Foundation for production-grade distributed storage (for example, DRBD-like systems).
- Reference implementation for kernel-to-userspace IPC communication patterns.
- Testbed for distributed consensus and replication algorithm research.

1.6 Key Project Specifications

Table 2. Key project specifications

Parameter	Value
Daemon source size	880 lines (<code>daemon/vdrsd_daemon.cpp</code>)
Kernel module source size	266 lines (<code>kernel/vdrsd.c</code>)
Chunk size	4 KB (4096-byte payload)
Write cache (RingBuffer) slots	512
Kernel ring buffer	256 KB (4096 x 64 = 262,144 bytes)
Cluster nodes	3 (ports 9001, 9002, 9003)

2 System Architecture

2.1 High-Level Architecture

VDRSD implements a layered, kernel-to-user communication model across three architectural tiers: user applications, the Linux kernel module, and a distributed cluster of user-space daemons. Figure 1 shows the overall structure.

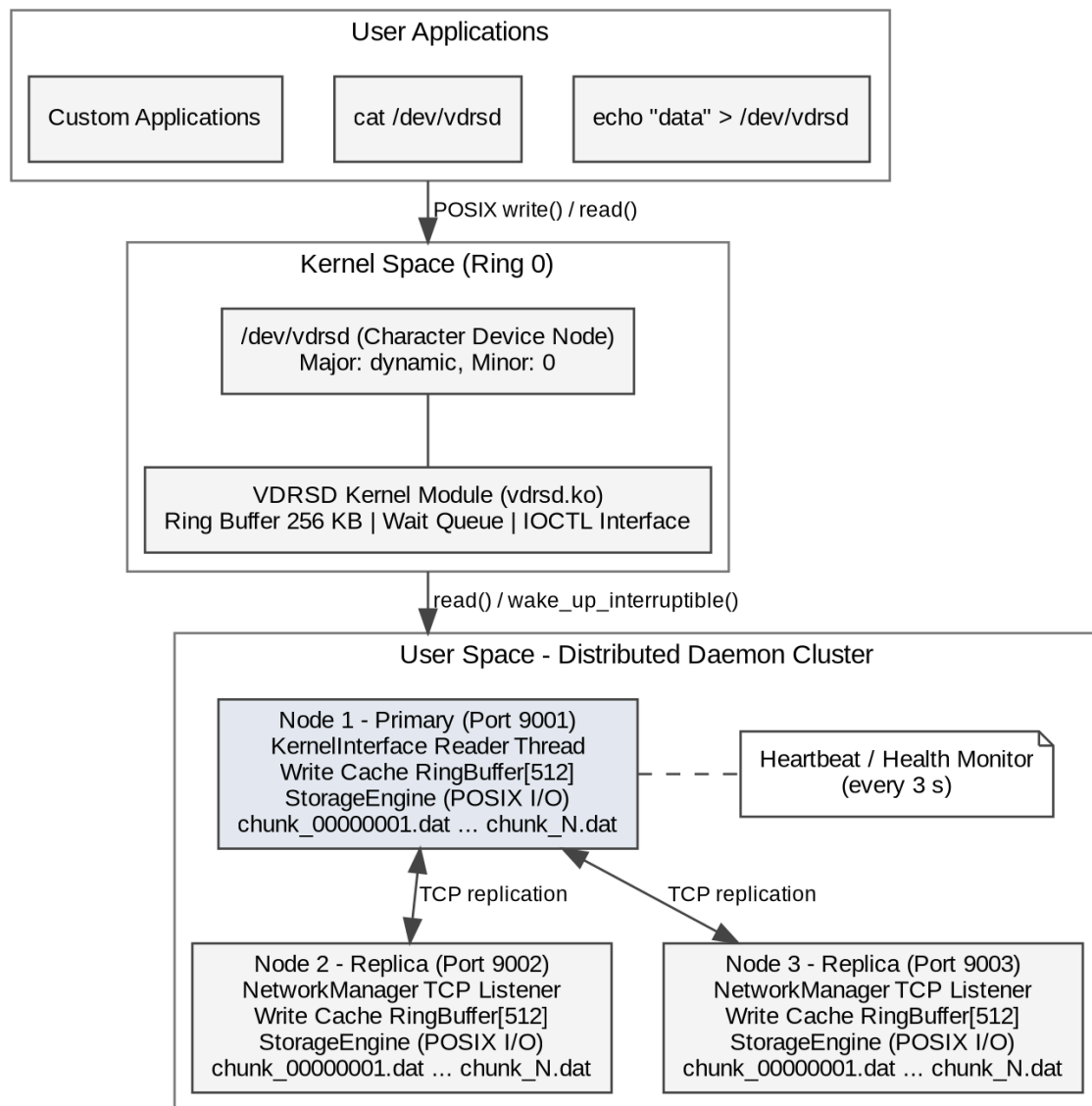


Figure 1. VDRSD three-tier architecture: user applications, Linux kernel module and distributed C++ daemon cluster

2.2 System Components

Table 3. System components

Component	Tier	Description
User applications	User space	Use POSIX <code>write()</code> and <code>read()</code> on <code>/dev/vdrsd</code> (for example <code>echo "data" > /dev/vdrsd</code> , <code>cat /dev/vdrsd</code> , or custom applications).
<code>/dev/vdrsd</code> character device node	Kernel space (Ring 0)	Device node with dynamically assigned major number and minor number 0.
VDRSD kernel module (<code>vdrsd.ko</code>)	Kernel space (Ring 0)	256 KB ring buffer, wait queue and IOCTL interface.
Node 1 (Primary, port 9001)	User space	<code>KernelInterface</code> reader thread, write cache <code>RingBuffer[512]</code> , and <code>StorageEngine</code> (POSIX I/O) storing <code>chunk_00000001.dat</code> to <code>chunk_N.dat</code> .
Node 2 and Node 3 (Replicas, ports 9002 and 9003)	User space	<code>NetworkManager</code> TCP listener, write cache <code>RingBuffer[512]</code> , and <code>StorageEngine</code> (POSIX I/O).
Heartbeat / health monitor	User space	Heartbeat messages to peers every 3 seconds.

2.3 Data Flow

Three kinds of traffic move through the system (see Figure 1):

- **Data flow:** user applications write through the VFS to `/dev/vdrsd`; the kernel module buffers the data; the daemon's reader thread reads it and stores it through the `StorageEngine`.
- **TCP replication:** each chunk is serialized as a `ReplicationMessage` and sent to every configured peer node.
- **Heartbeat:** the health monitor periodically probes all peers (every 3 seconds) to track node liveness.

For reads, data can be retrieved from local storage or from any live replica node.

2.4 Architectural Pipeline

A write travels through the following stages, from the user application to persistent storage on every node.

1. **User application initiates write.** An application invokes `write()` on the file descriptor opened against `/dev/vdrsd`, passing raw data bytes.
2. **VFS routes to the kernel module.** The Linux VFS layer routes the call to `vdrsd_write()`, registered in the `struct file_operations` table.
3. **Kernel buffers data in the ring buffer.** The driver copies data from user space via `copy_from_user()`, writes it into the 256 KB circular ring buffer protected by a spinlock, and calls `wake_up_interruptible()`.

4. **Daemon reads a DataChunk.** The `KernelInterface` reader thread, blocked in `read()` on `/dev/vdrsd`, wakes up, reads a 4 KB payload, and constructs a `DataChunk` with a CRC32 checksum and a nanosecond timestamp.
5. **Asynchronous local storage.** The chunk is pushed to the `RingBuffer<DataChunk, 512>` write cache. A background flush thread pulls from the cache and calls POSIX `write()` and `fsync()` to persist `chunk_XXXXXXXX.dat`.
6. **TCP replication to peer nodes.** `NetworkManager::replicate_chunk()` opens a TCP connection to each configured peer, sends a serialized `ReplicationMessage` (type `DATA_WRITE`, sequence number, chunk), and marks the peer ONLINE or OFFLINE based on success.
7. **Peer stores replica.** Each peer's `handle_client()` receives the message, verifies the CRC32 checksum, and passes the chunk to its own `StorageEngine` for local persistence.

The final stage of the pipeline, health monitoring and catch-up synchronization, is described in Section 2.5.

2.5 Fault-Tolerance and Recovery Flow

The `HealthMonitor` sends heartbeat messages to all peers every 3 seconds. When a node transitions from OFFLINE to ONLINE, the `ClusterSyncObserver` triggers a `SYNC_REQUEST` to replay missing chunks. The recovery sequence is:

1. A peer fails; replication to it is reported as OFFLINE and the node continues in degraded mode with the remaining replicas.
2. When the peer returns, the `HealthMonitor` detects the OFFLINE to ONLINE transition and notifies the registered observers.
3. The `ClusterSyncObserver` marks the peer SYNCING, reads the latest persisted `chunk_id` from its `StorageEngine`, and sends a `SYNC_REQUEST` carrying that ID and the local port.
4. The remote peer pushes the missing chunks back using `DATA_WRITE` messages, restoring full replication.

The implementation of this flow is covered in Section 6.9, and its observed behavior in Section 10.

3 Requirements

3.1 Functional Requirements

Table 4. Functional requirements

ID	Requirement	Priority
FR-01	Character device <code>/dev/vdrsd</code> accepts <code>open</code> , <code>read</code> , <code>write</code> , <code>release</code> and <code>ioctl</code> operations.	High
FR-02	Kernel module buffers write data (256 KB ring buffer) and signals the user-space daemon via <code>wake_up_interruptible()</code> .	High
FR-03	C++ daemon runs as a background Linux daemon (<code>fork</code> , <code>setuid</code> , signal handling).	High
FR-04	Daemon receives data from the kernel via the character device read interface.	High
FR-05	Daemon replicates received data to peer nodes via TCP sockets with CRC32 integrity verification.	High
FR-06	Daemon persists data to the local filesystem using POSIX file I/O (<code>write</code> , <code>fsync</code>).	High
FR-07	Read requests retrieve data from local storage or replicas.	Medium
FR-08	Node health monitoring via a heartbeat mechanism with Observer pattern notification.	Medium
FR-09	Configurable cluster topology via the <code>vdrsd.conf</code> file (Singleton <code>ConfigManager</code>).	Low
FR-10	Comprehensive logging: <code>printk</code> for the kernel, <code>std::cout/std::cerr</code> for the daemon.	Medium

3.2 Non-Functional Requirements

Table 5. Non-functional requirements

ID	Requirement	Target
NFR-01	Replication latency	< 100 ms on localhost cluster
NFR-02	Data consistency model	Eventual consistency across replicas
NFR-03	Reliability	System tolerates single-node failure
NFR-04	Code quality	SOLID principles, Valgrind-clean (no memory leaks)
NFR-05	Portability	Linux kernel 5.x+ with GCC 11+ / C++17
NFR-06	Documentation	Complete inline docs and external project report

3.3 Development Timeline

The project followed a six-week, six-stage plan. The corresponding Gantt chart is shown in Section 13.4.

Table 6. Development timeline

Week	Stage	Key activities
Week 1	Stage 1-2	Project introduction, requirements, PRD, development plan
Week 2	Stage 3	System design, architecture diagrams, UML, environment setup
Week 3	Stage 4	Core implementation: kernel module and daemon skeleton
Week 4	Stage 4-5	Networking, replication, integration with kernel interface
Week 5	Stage 5	Testing, debugging, performance improvement
Week 6	Stage 6	Final integration, documentation, presentation

4 System Design

4.1 Core Data Structures

Three structures form the core of the daemon design: the data unit exchanged between kernel, daemon and peers, the cluster node state, and the wire protocol message.

DataChunk

DataChunk is the core data unit passed between the kernel, user space and peer nodes (4136 bytes total, as specified in the source design).

Listing 1. DataChunk structure

```
// Core data chunk: kernel <-> user-space <-> peer nodes (4136 bytes total)
struct DataChunk {
    uint64_t chunk_id;        // Monotonically increasing unique ID
    uint64_t timestamp;       // Write timestamp (nanoseconds since epoch)
    uint32_t size;            // Actual payload bytes written (<= 4096)
    uint32_t checksum;        // CRC32 of payload for integrity verification
    uint8_t  payload[4096];   // Data payload (page-aligned)
};
```

PeerNode

PeerNode describes the state of a cluster node as seen by the local daemon.

Listing 2. NodeStatus and PeerNode structures

```
enum class NodeStatus { ONLINE, OFFLINE, SYNCING };

struct PeerNode {
    std::string node_id;      // "node1", "node2", "node3"
    std::string ip_address;   // IPv4 address string
    uint16_t    port;         // TCP listen port
    NodeStatus  status;       // Current health state
    std::chrono::steady_clock::time_point last_heartbeat;
};
```

ReplicationMessage

ReplicationMessage is the wire protocol unit. The MessageType field discriminates the six message kinds.

Listing 3. MessageType and ReplicationMessage structures

```
enum class MessageType : uint8_t {
    DATA_WRITE = 1, // Chunk replication to peer
    DATA_READ  = 2, // Read request to replica
    HEARTBEAT   = 3, // Liveness probe
    ACK         = 4, // Positive acknowledgement
    NACK        = 5, // Negative acknowledgement
    SYNC_REQUEST = 6 // Catch-up sync after recovery
};

struct ReplicationMessage {
    MessageType type; // Wire type discriminator
    uint64_t    sequence_num; // Chunk ID for dedup / ordering
    DataChunk   chunk;        // Embedded payload (4136 bytes)
};
```

4.2 Component Responsibilities

Table 7. Component responsibility matrix

Component	Class / file	Responsibility
Kernel driver	<code>kernel/vdrsd.c</code>	Expose <code>/dev/vdrsd</code> , buffer data (256 KB ring), signal the daemon.
Kernel interface	<code>KernelInterface</code>	Open <code>/dev/vdrsd</code> , run the reader thread, feed <code>StorageEngine</code> and <code>NetworkManager</code> .
Storage engine	<code>StorageEngine</code>	Asynchronous write cache (<code>RingBuffer[512]</code>), POSIX flush, chunk directory scan.
Network manager	<code>NetworkManager</code>	TCP server (accept loop), client connections, <code>replicate_chunk()</code> , <code>sync_peer()</code> .
Health monitor	<code>HealthMonitor</code>	Heartbeat loop (3 s), Observer notification on status change.
Cluster sync observer	<code>ClusterSyncObserver</code>	Triggers <code>SYNC_REQUEST</code> when a peer comes back ONLINE.
Config manager	<code>ConfigManager</code>	Singleton: parses <code>vdrsd.conf</code> , exposes <code>node_id</code> , port and peer list.
Daemon lifecycle	<code>Daemon</code>	Signal handler (SIGTERM/SIGINT), graceful shutdown coordination.

4.3 Design Patterns

Table 8. Design patterns applied

Pattern	Applied in	Purpose
Singleton	<code>ConfigManager::get_instance()</code>	One configuration object, loaded once and accessed everywhere.
Observer	<code>INodeObserver / HealthMonitor</code>	<code>ClusterSyncObserver</code> reacts to ONLINE/OFFLINE status transitions.
Producer-Consumer	<code>RingBuffer<T, N></code>	Decouples fast writers from slower disk flushers using condition variables.
RAII	<code>device_fd_, server_fd_</code>	All file descriptors are closed deterministically in destructors.

4.4 UML Class Diagram

Figure 2 shows the main classes of the daemon and their relationships. The `VDRSDDaemon` class owns the network, storage, kernel-interface and health-monitor subsystems and uses the singleton `ConfigManager`; `HealthMonitor` notifies observers through the `INodeObserver` interface.

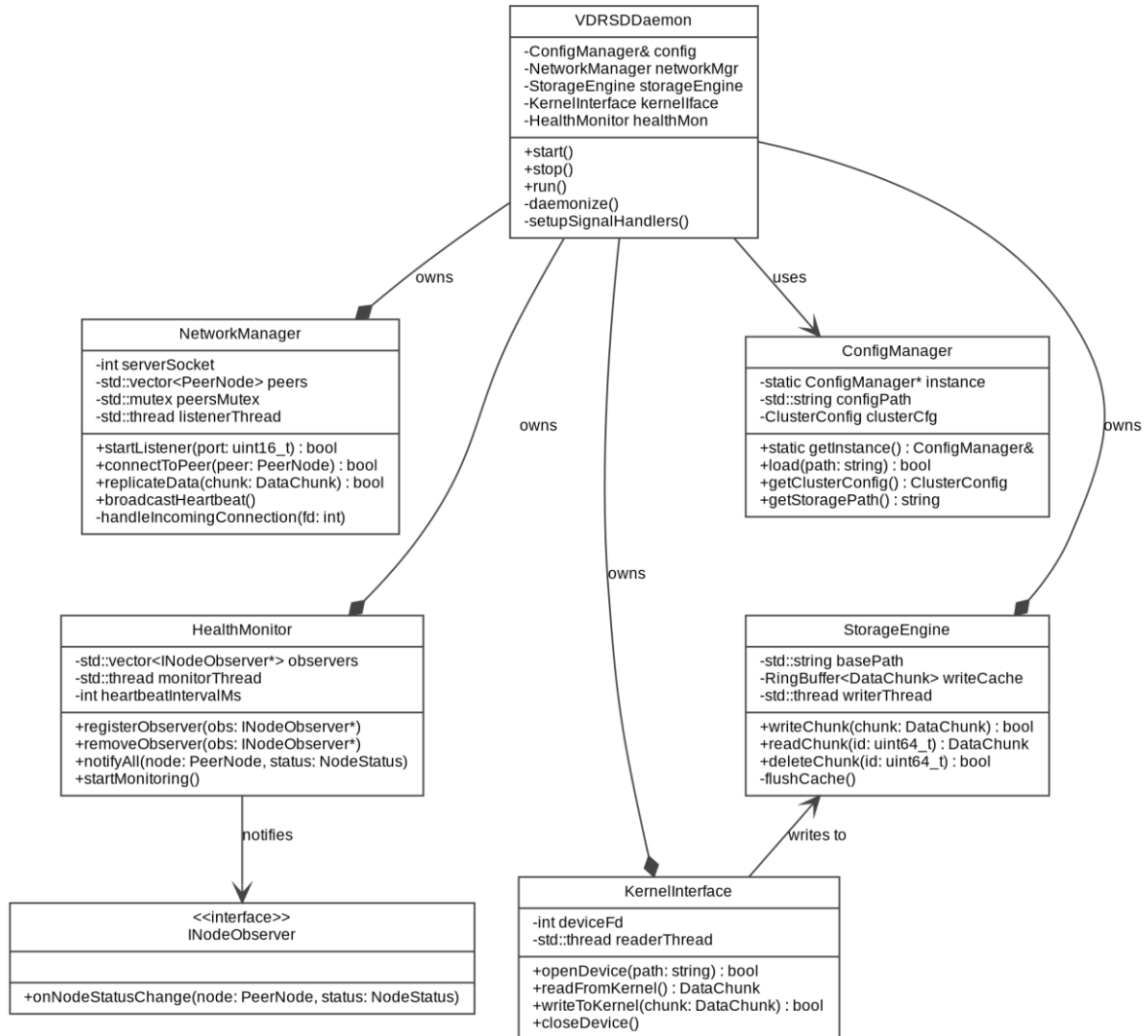


Figure 2. UML class diagram of the VDRSD daemon (generated from `vdrsd_daemon.cpp`)

4.5 Sequence Diagram

Figure 3 shows the data write flow from the user application through the kernel module and daemon to the peers. After the daemon reads the chunk, local persistence and peer replication proceed in parallel; the write returns to the caller once the operations are acknowledged.

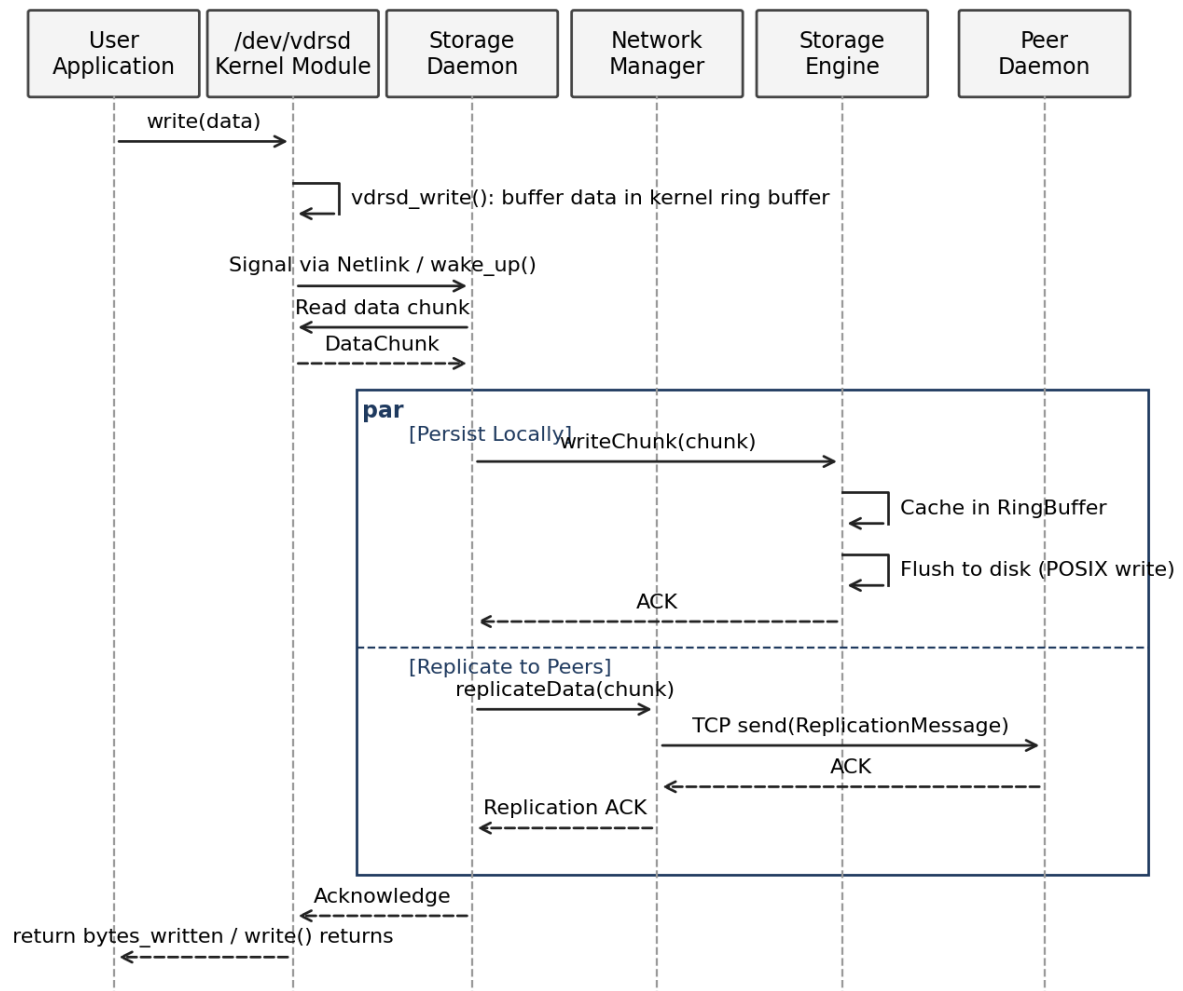


Figure 3. Sequence diagram of the data write flow (user, kernel, daemon, peers)

4.6 State Machine

Figure 4 shows the lifecycle of the daemon. Start-up proceeds through configuration loading, kernel connection and peer discovery before reaching the Running state. A peer failure moves the daemon to Degraded mode, and a termination signal (SIGTERM or SIGINT) begins an orderly shutdown.

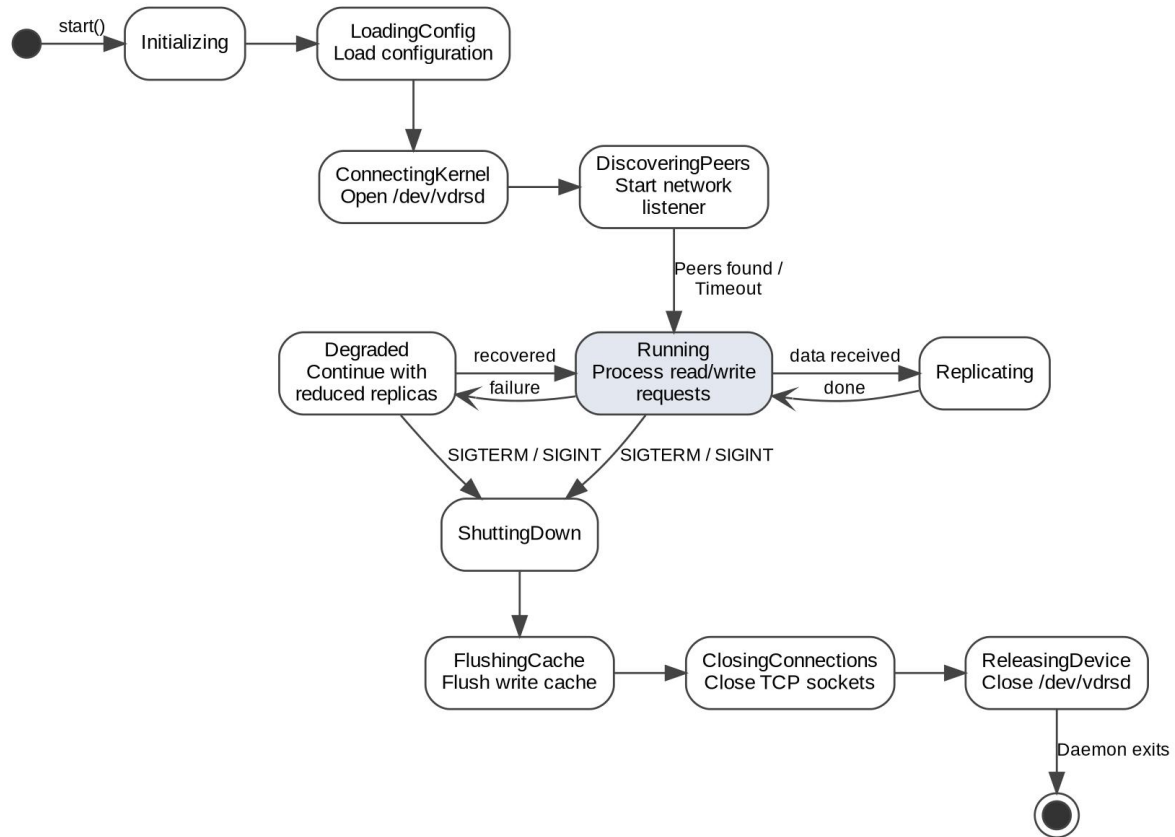


Figure 4. State machine of the VDRSD daemon lifecycle

5 Kernel Module: vdrsd.c

5.1 Overview

The kernel module (`kernel/vdrsd.c`, 266 lines) provides the `/dev/vdrsd` character device and all associated file operations. It runs entirely in kernel space (Ring 0) and is built as `vdrsd.ko`.

Note. Kernel-space code runs with full hardware privileges, so any crash in this module is a kernel panic. All memory allocations use `GFP_KERNEL` and are freed in the cleanup path. Spinlocks are always paired with their unlock counterparts.

5.2 Module Metadata

Table 9. Kernel module metadata

Field	Value
Module file	<code>kernel/vdrsd.c</code>
Output	<code>vdrsd.ko</code>
License	GPL
Author	Wipro Embedded Track
Description	Virtual Distributed Replicated Storage Device Driver
Version	1.0
Buffer size	4096 x 64 = 262,144 bytes (256 KB)
Device name	<code>/dev/vdrsd</code>
Class name	<code>vdrsd_class</code>
IOCTL magic	'v'

5.3 Character Device Interface

The driver registers the device name `vdrsd` (macro `VDRSD_DEVICE_NAME`) with a dynamically allocated major number, minor number 0, and a device class `vdrsd_class`, so that the node `/dev/vdrsd` is created automatically on module load. The driver state is held in the following static variables: `major_number`, `vdrsd_class`, `vdrsd_device` and the `cdev` structure `vdrsd_cdev`.

The device supports the `open`, `read`, `write`, `release` and `ioctl` operations (requirement FR-01), registered through the `struct file_operations` table:

- `vdrsd_write()` allocates a temporary buffer with `kmalloc(len, GFP_KERNEL)`, copies the data from user space with `copy_from_user()` (returning `-ENOMEM` or `-EFAULT` on failure), places it in the ring buffer, and wakes the reader.
- `vdrsd_read()` sleeps in `wait_event_interruptible()` until the buffer holds data (`count > 0`), then copies it to user space with `copy_to_user()`.

5.4 Ring Buffer

Written data is held in a circular queue of `VDRSD_BUF_SIZE = 4096 x 64 bytes (256 KB)` allocated as `kernel_buffer`. The queue is managed with three indices and protected by a spinlock; readers sleep on a wait queue until data is available.

Table 10. Kernel ring buffer state

Variable	Purpose
<code>kernel_buffer</code>	Circular byte buffer of 256 KB.
<code>head, tail, count</code>	Read position, write position, and number of bytes currently stored.
<code>buffer_lock</code>	Spinlock protecting the buffer and statistics.
<code>read_wait_queue</code>	Wait queue (<code>DECLARE_WAIT_QUEUE_HEAD</code>) on which readers sleep.
<code>driver_stats</code>	Instance of <code>struct vdrsd_stats</code> holding the driver statistics.

On a write, the driver computes the free space (`VDRSD_BUF_SIZE - count`), copies at most that many bytes into the buffer (advancing `tail` modulo the buffer size), updates the counters and statistics under the spinlock, and finally calls `wake_up_interruptible()` to signal the daemon.

5.5 IOCTL Interface

Table 11. IOCTL commands

Command	Code	Description
<code>VDRSD_IOCTL_GET_STATS</code>	<code>_IOR('v', 1, struct vdrsd_stats)</code>	Returns total writes/reads, bytes transferred and current buffer utilization.
<code>VDRSD_IOCTL_CLEAR_BUFFER</code>	<code>_IO('v', 2)</code>	Resets the ring buffer head/tail/count pointers atomically under the spinlock.

5.6 Statistics Structure

Listing 4. `struct vdrsd_stats`

```
struct vdrsd_stats {
    u64 total_writes;    // Total write() calls served
    u64 total_reads;     // Total read() calls served
    u64 bytes_written;   // Cumulative bytes written
    u64 bytes_read;      // Cumulative bytes read
    u32 buffer_used;     // Current bytes in ring buffer
    u32 buffer_capacity; // Ring buffer capacity (262144)
};
```

5.7 Important Kernel Functions

Table 12. Important kernel functions and macros

Function / macro	Role
<code>vdrsd_write()</code>	Handles <code>write()</code> : copies user data into the ring buffer and wakes the daemon.
<code>vdrsd_read()</code>	Handles <code>read()</code> : blocks until data is available, then copies it to user space.
<code>copy_from_user()</code> / <code>copy_to_user()</code>	Safe data transfer between user space and kernel space.
<code>kmalloc()</code> / <code>kfree()</code>	Temporary buffer allocation (<code>GFP_KERNEL</code>) and release.
<code>spin_lock()</code> / <code>spin_unlock()</code>	Protect the ring buffer indices and statistics.
<code>wake_up_interruptible()</code>	Wakes the user-space daemon blocked on <code>read_wait_queue</code> .
<code>wait_event_interruptible()</code>	Puts the reader to sleep until <code>count > 0</code> .
<code>module_init(vdrsd_init)</code> / <code>module_exit(vdrsd_exit)</code>	Module entry and exit points.

Source listing

The following excerpt shows the header, global state, `vdrsd_write()`, the beginning of `vdrsd_read()`, and the module metadata macros.

Listing 5. Excerpt of `kernel/vdrsd.c`

```
/*
 * vdrsd.c - Linux Character Device Driver for VDRSD
 *
 * Provides /dev/vdrsd device interface for user application writes,
 * buffers data in kernel ring buffer, and signals user-space daemon.
 */
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/fs.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/uaccess.h>
#include <linux/slab.h>
#include <linux/spinlock.h>
#include <linux/wait.h>
#include <linux/ioctl.h>
#include <linux/version.h>

#define VDRSD_DEVICE_NAME "vdrsd"
#define VDRSD_BUF_SIZE (4096 * 64)    // 256 KB Kernel Ring Buffer

static int major_number;
static struct class *vdrsd_class = NULL;
static struct device *vdrsd_device = NULL;
static struct cdev vdrsd_cdev;

// Kernel Ring Buffer - head/tail circular queue
static char *kernel_buffer;
static size_t head = 0, tail = 0, count = 0;
static spinlock_t buffer_lock;
```

```

static DECLARE_WAIT_QUEUE_HEAD(read_wait_queue);
static struct vdrsd_stats driver_stats = {0};

static ssize_t vdrsd_write(struct file *filep, const char __user *buffer,
                          size_t len, loff_t *offset) {
    char *temp_buf = kmalloc(len, GFP_KERNEL);
    if (!temp_buf) return -ENOMEM;
    if (copy_from_user(temp_buf, buffer, len)) {
        kfree(temp_buf);
        return -EFAULT;
    }

    spin_lock(&buffer_lock);
    size_t avail = VDRSD_BUF_SIZE - count;
    size_t to_write = min(len, avail);
    for (size_t i = 0; i < to_write; i++) {
        kernel_buffer[tail] = temp_buf[i];
        tail = (tail + 1) % VDRSD_BUF_SIZE;
    }
    count += to_write;
    driver_stats.bytes_written += to_write;
    driver_stats.total_writes++;
    spin_unlock(&buffer_lock);

    kfree(temp_buf);
    wake_up_interruptible(&read_wait_queue);    // Signal daemon
    return to_write;
}

static ssize_t vdrsd_read(struct file *filep, char __user *buffer,
                          size_t len, loff_t *offset) {
    wait_event_interruptible(read_wait_queue, count > 0);
    // ... copy data to user space via copy_to_user() ...
}

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Wipro Embedded Track");
MODULE_DESCRIPTION("Virtual Distributed Replicated Storage Device Driver");
MODULE_VERSION("1.0");
module_init(vdrsd_init);
module_exit(vdrsd_exit);

```

5.8 Kernel Version Compatibility

The driver handles the Linux 6.4 API change in which `class_create()` no longer takes `THIS_MODULE` as a parameter. A compile-time version check selects the correct call:

```
#if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 4, 0)
```

The project targets Linux kernel 5.x and later (NFR-05) and was demonstrated on a 6.8.0-49-generic kernel (Section 10.1).

6 User-Space Daemon: vdrsd_daemon.cpp

6.1 Overview

The daemon is an 880-line, self-contained C++17 service with zero external header dependencies. All subsystems are contained in a single compilation unit for educational clarity. It runs as a background Linux daemon (fork, setsid, signal handling) and comprises the following components:

- **ConfigManager**: Singleton that parses `vdrsd.conf`.
- **KernelInterface**: reads data from `/dev/vdrsd`.
- **StorageEngine**: asynchronous write cache and POSIX persistence.
- **NetworkManager**: TCP server, replication and peer synchronization.
- **HealthMonitor** and **ClusterSyncObserver**: heartbeat, Observer notification and catch-up sync.
- **RingBuffer<T, N>** and **CRC32** utilities shared by the subsystems.

6.2 Thread Architecture

The daemon consists of a main thread, four subsystem threads and any number of detached client-handler threads (Figure 5).

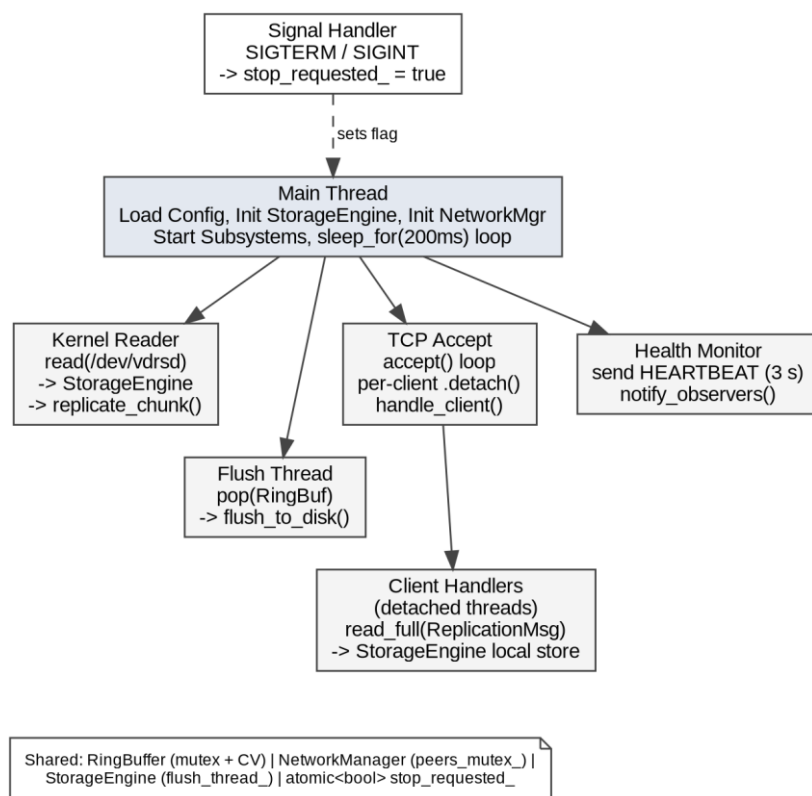


Figure 5. Daemon thread architecture: main thread, four subsystem threads and N detached client handlers

Table 13. Daemon threads

Thread	Behavior
Main thread	Loads configuration, initializes <code>StorageEngine</code> and <code>NetworkManager</code> , starts the subsystems, then loops with <code>sleep_for(200ms)</code> until stop is requested.
Kernel reader	Blocks in <code>read(/dev/vdrsd)</code> ; passes each chunk to <code>StorageEngine</code> and calls <code>replicate_chunk()</code> .
Flush thread	Pops chunks from the <code>RingBuffer</code> and calls <code>flush_to_disk()</code> .
TCP accept	Runs the <code>accept()</code> loop and detaches a thread per client running <code>handle_client()</code> .
Health monitor	Sends <code>HEARTBEAT</code> messages every 3 seconds and calls <code>notify_observers()</code> .
Client handlers	Detached threads: <code>read_full(ReplicationMsg)</code> then local store through <code>StorageEngine</code> .
Signal handler	On <code>SIGTERM/SIGINT</code> sets <code>stop_requested_ = true</code> .

Shared state is synchronized as follows: the `RingBuffer` uses a mutex and condition variables, `NetworkManager` protects its peer list with `peers_mutex_`, `StorageEngine` owns `flush_thread_`, and shutdown is coordinated through the `atomic<bool> stop_requested_flag`.

6.3 Kernel Interface

`KernelInterface` opens the character device `/dev/vdrsd` and runs a reader thread. The thread blocks in `read()` until the kernel module signals new data, reads a 4 KB payload, and constructs a `DataChunk` with a monotonically increasing chunk ID, a CRC32 checksum and a nanosecond timestamp. Each chunk is handed to the `StorageEngine` for local persistence and to the `NetworkManager` for replication. On start-up the daemon logs the next chunk ID it will use.

6.4 Storage Engine

`StorageEngine` decouples the reader from slow disk I/O. Incoming chunks are pushed to a `RingBuffer<DataChunk, 512>` write cache. A background flush thread pops chunks and persists each one with `POSIX write()` followed by `fsync()` as `chunk_XXXXXXXX.dat` (an eight-digit chunk number) under the node's storage path, for example `node1_data/chunk_00000001.dat`. The engine also scans the chunk directory and reports the latest persisted chunk ID through `get_latest_chunk_id()`, which is used for catch-up synchronization.

6.5 Network Manager

`NetworkManager` implements the TCP server and client sides of replication:

- A TCP server runs an `accept()` loop; each client is served by a detached thread running `handle_client()`, which reads a full `ReplicationMessage`, verifies its CRC32 checksum and passes the chunk to the local `StorageEngine`.

- `replicate_chunk()` opens a TCP connection to each configured peer, sends the serialized `ReplicationMessage` of type `DATA_WRITE`, and marks the peer `ONLINE` or `OFFLINE` based on success.
- `send_message()` sends an arbitrary message to a peer address, and `sync_peer()` pushes missing chunks to a recovering peer.
- The peer list is protected by `peers_mutex_`; peer states (`ONLINE`, `OFFLINE`, `SYNCING`) are changed with `set_peer_status()`.

6.6 Health Monitor

`HealthMonitor` runs its own thread that sends `HEARTBEAT` messages to all peers every 3 seconds. When a peer's status changes, the monitor notifies all registered observers through `INodeObserver` (Observer pattern). The `ClusterSyncObserver` described in Section 6.9 is the observer used for fault recovery.

6.7 Ring Buffer

`RingBuffer<T, Capacity>` is a thread-safe, blocking or non-blocking producer-consumer queue built on `std::mutex` and two condition variables. The default capacity is 512 slots. A blocking

`push()` waits while the buffer is full and a blocking `pop()` waits while it is empty; with `block = false` both return false immediately instead of waiting.

Listing 6. RingBuffer template

```
// Thread-safe, blocking/non-blocking producer-consumer ring buffer
template<typename T, size_t Capacity = 512>
class RingBuffer {
private:
    std::vector<T> buffer_;
    size_t head_{0}, tail_{0}, size_{0};
    mutable std::mutex mutex_;
    std::condition_variable not_full_;
    std::condition_variable not_empty_;
public:
    RingBuffer() : buffer_(Capacity) {}

    // Blocking push - waits if full (default) or returns false immediately
    bool push(const T& item, bool block = true) {
        std::unique_lock<std::mutex> lock(mutex_);
        if (block)
            not_full_.wait(lock, [this] { return size_ < Capacity; });
        else if (size_ == Capacity) return false;
        buffer_[tail_] = item;
        tail_ = (tail_ + 1) % Capacity;
        ++size_;
        not_empty_.notify_one();
        return true;
    }

    // Blocking pop - waits if empty (default) or returns false immediately
    bool pop(T& item, bool block = true) {
        std::unique_lock<std::mutex> lock(mutex_);
        if (block)
            not_empty_.wait(lock, [this] { return size_ > 0; });
        else if (size_ == 0) return false;
        item = buffer_[head_];
        head_ = (head_ + 1) % Capacity;
        --size_;
        not_full_.notify_one();
        return true;
    }
};
```

6.8 CRC32 Integrity

Every chunk carries a CRC32 checksum of its payload, computed in software with the Ethernet polynomial 0xEDB88320 (slicing-by-1). The checksum is verified on every replica before the chunk is stored.

Listing 7. CRC32 implementation

```
// Software CRC32 - Slicing-by-1 (Ethernet polynomial 0xEDB88320)
inline uint32_t calculate_crc32(const uint8_t* data, size_t len) {
    uint32_t crc = 0xFFFFFFFF;
    for (size_t i = 0; i < len; ++i) {
        crc ^= data[i];
        for (int j = 0; j < 8; ++j) {
            uint32_t mask = -(crc & 1);
            crc = (crc >> 1) ^ (0xEDB88320 & mask);
        }
    }
    return ~crc;
}
```

6.9 Cluster Synchronization

ClusterSyncObserver implements INodeObserver. When a peer returns ONLINE, it requests catch-up synchronization in four steps, as marked in the listing below.

Listing 8. ClusterSyncObserver

```
class ClusterSyncObserver : public INodeObserver {
public:
    void on_node_status_change(const PeerNode& node,
                               NodeStatus new_status) override {
        if (new_status == NodeStatus::ONLINE) {
            // 1. Mark SYNCING
            network_mgr_.set_peer_status(node.node_id, NodeStatus::SYNCING);
            // 2. Query our latest persisted chunk_id
            uint64_t latest_id = storage_engine_.get_latest_chunk_id();
            // 3. Send SYNC_REQUEST carrying our latest chunk_id + local port
            ReplicationMessage sync_req;
            sync_req.type = MessageType::SYNC_REQUEST;
            sync_req.sequence_num = latest_id;
            sync_req.chunk.size = local_port_; // encoded in size field
            network_mgr_.send_message(node.ip_address, node.port, sync_req);
            // 4. Remote peer will push missing chunks back via DATA_WRITE
        }
    }
};
```

6.10 Graceful Shutdown

A SIGTERM or SIGINT signal sets the atomic flag `stop_requested_`, which the main thread observes in its 200 ms loop. Shutdown then proceeds in order:

1. Flush the write cache to disk (StorageEngine).
2. Close TCP sockets and stop the network threads (NetworkManager).
3. Stop the heartbeat loop (HealthMonitor).
4. Close `/dev/vdrsd` and exit the daemon.

File descriptors such as `device_fd_` and `server_fd_` are released deterministically in destructors (RAII). The expected log output is shown in Section 10.7.

7 Build System and Project Structure

7.1 Required Tools

Table 14. Build toolchain requirements

Tool	Version	Purpose
gcc / g++	11+	C and C++17 compilation
make	4.x	Build orchestration (kbuild and daemon)
linux-headers	<code>\$(uname -r)</code>	Kernel module compilation against the running kernel
pthread	POSIX	Multithreading support for the daemon
valgrind	3.x	Memory leak detection (user space)
cppcheck	2.x	Static analysis

7.2 Installation

Install all required packages on Ubuntu or Debian:

```
# Install all required packages (Ubuntu / Debian)
sudo apt update
sudo apt install -y build-essential linux-headers-$(uname -r) \
    g++ make valgrind cppcheck clang-format git
```

7.3 Root Makefile

The root Makefile compiles the daemon, delegates the kernel module build to kbuild, and provides targets to run the tests and clean the tree.

Listing 9. Root Makefile

```
# VDRSD Root Makefile
CXX = g++
CXXFLAGS = -std=c++17 -Wall -Wextra -pthread
DAEMON_SRC = daemon/vdrsd_daemon.cpp
TARGET = vdrsd_daemon

all: $(TARGET) tests_all kernel_module

# Compile the user-space daemon
$(TARGET): $(DAEMON_SRC)
    $(CXX) $(CXXFLAGS) -o $@ $<

# Delegate kernel module build to kbuild
kernel_module:
    $(MAKE) -C kernel

# Run all unit + integration + stress tests
run_tests: tests_all
    @./tests/unit/test_ring_buffer
    @./tests/unit/test_storage_engine
    @./tests/unit/test_network
    @./tests/integration/test_replication
    @./tests/integration/test_recovery

clean:
    rm -f $(TARGET) tests/unit/test_* tests/integration/test_* \
        tests/stress/stress_write
    rm -rf vdrsd_data test_data node*_data stress_data
    $(MAKE) -C kernel clean || true

.PHONY: all tests_all run_tests kernel_module clean
```

7.4 Kernel Makefile

Listing 10. kernel/Makefile

```
# kernel/Makefile - delegates to kbuild
obj-m += vdrsd.o
KDIR = /lib/modules/$(shell uname -r)/build
PWD = $(shell pwd)

all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
```

7.5 Project Directory Structure

Listing 11. Project directory structure

```
VDRSD/
├── Makefile                                # Root build orchestrator
├── README.md                              # Complete project documentation
├── kernel/
│   ├── Makefile                            # kbuild wrapper
│   └── vdrsd.c                             # Character device driver (266 lines)
├── daemon/
│   └── vdrsd_daemon.cpp                    # All-in-one C++17 daemon (880 lines)
├── config/
│   ├── vdrsd.conf                         # Default node config
│   ├── node1.conf                         # Node 1 - port 9001
│   └── node2.conf                         # Node 2 - port 9002
├── scripts/
│   ├── start.sh                           # Load module + start daemon cluster
│   ├── stop.sh                            # Stop daemons + unload module
│   └── status.sh                          # Check running daemon + dmesg
├── tests/
│   ├── unit/
│   │   ├── test_ring_buffer.cpp
│   │   ├── test_storage_engine.cpp
│   │   └── test_network.cpp
│   ├── integration/
│   │   ├── test_replication.cpp
│   │   └── test_recovery.cpp
│   └── stress/
│       └── stress_write.cpp
├── docs/
│   ├── project_report.md                  # Source documentation
│   ├── vdrsd_documentation.html          # HTML documentation
│   └── images/
│       ├── system_architecture.svg
│       ├── class_diagram.svg
│       ├── sequence_diagram.svg
│       ├── state_machine.svg
│       ├── git_branching_strategy.svg
│       └── project_timeline_gantt.svg
```

7.6 Build Commands

Table 15. Build and run commands

Command	Description
make	Builds the kernel module (<code>vdrsd.ko</code>) and the user-space daemon (<code>vdrsd_daemon</code>).
make kernel_module	Builds only the kernel module, by delegating to <code>kernel/Makefile</code> .
make run_tests	Runs the unit and integration test executables.
make clean	Removes build outputs and test data directories.
sudo insmod kernel/vdrsd.ko	Loads the kernel module.
sudo rmmod vdrsd	Unloads the kernel module.

The daemon is compiled with the command `g++ -std=c++17 -Wall -Wextra -pthread -o vdrsd_daemon daemon/vdrsd_daemon.cpp`.

8 Configuration and Cluster Setup

8.1 Configuration File Format

The daemon reads an INI-style configuration file. Lines starting with # are comments, key-value pairs use = as the delimiter, and peer entries use the colon-separated `id:ip:port` format.

Listing 12. Configuration file (`vdrsd.conf`)

```
#
# VDRSD Configuration File - /config/vdrsd.conf
#
node_id      = node1
listen_port  = 9001
storage_path = ./vdrsd_data
device_path  = /dev/vdrsd

# Peer list (format: peer=id:ip:port)
peer = node2:127.0.0.1:9002
peer = node3:127.0.0.1:9003
```

8.2 Cluster Topology

Table 16. Cluster topology

Node	Role	Port	Config file	Storage path
node1	Primary (writes to kernel device)	9001	config/node1.conf	./node1_data/
node2	Replica (receives TCP replication)	9002	config/node2.conf	./node2_data/
node3	Replica (receives TCP replication)	9003	config/vdrsd.conf	./vdrsd_data/

8.3 Node Configuration

Each daemon is started with one configuration file. `ConfigManager` (a Singleton) parses it once and exposes the node ID, listen port, storage path and peer list to the other subsystems.

Table 17. Configuration keys

Key	Meaning	Example
<code>node_id</code>	Identifier of this node	node1
<code>listen_port</code>	TCP port on which this node listens	9001
<code>storage_path</code>	Directory where chunk files are stored	./vdrsd_data
<code>device_path</code>	Path of the kernel character device	/dev/vdrsd
<code>peer</code>	Peer entry in <code>id:ip:port</code> format (repeatable)	node2:127.0.0.1:9002

All nodes run on the local host (127.0.0.1) and are distinguished by their ports 9001, 9002 and 9003.

8.4 Startup Script

`scripts/start.sh` loads the kernel module and starts all three daemons:

Listing 13. `scripts/start.sh`

```
# scripts/start.sh - Load kernel module and start all three daemons
sudo insmod kernel/vdrsd.ko
sleep 1
sudo chmod 666 /dev/vdrsd
./vdrsd_daemon config/node1.conf &
./vdrsd_daemon config/node2.conf &
./vdrsd_daemon config/vdrsd.conf &
echo "VDRSD cluster started on ports 9001, 9002, 9003"
```

8.5 Shutdown Script

`scripts/stop.sh` stops the daemons and unloads the kernel module. Sending SIGTERM to a daemon triggers the graceful shutdown sequence described in Section 6.10. The companion script `scripts/status.sh` checks the running daemons and the kernel log (`dmesg`). The output observed when stopping the cluster is shown in Section 10.7.

9 Testing

9.1 Testing Strategy

Testing combines four complementary approaches, from individual classes up to whole-cluster behavior and memory safety.

Table 18. Testing strategy

Test type	Scope	Tools	Coverage
Unit testing	Individual classes and functions	Custom C++ assertions	RingBuffer, StorageEngine, CRC32, NetworkManager serialization
Integration testing	Module interactions	C++ test harness	Kernel to daemon pipeline, daemon to peer replication, recovery sync
Stress testing	Performance under load	<code>stress_write.cpp</code>	High-frequency writes, concurrent connections, buffer overflow behavior
Memory testing	Leak detection	Valgrind (user space), KASAN (kernel)	No leaks under all shutdown and error paths

9.2 Unit Testing

Unit tests use custom C++ assertions and are located in `tests/unit/`:

- `test_ring_buffer.cpp`: the thread-safe RingBuffer.
- `test_storage_engine.cpp`: the StorageEngine and its chunk files.
- `test_network.cpp`: NetworkManager serialization.

CRC32 is also covered at unit level. Static analysis is performed with `cppcheck`.

9.3 Integration Testing

Integration tests are located in `tests/integration/` (`test_replication.cpp` and `test_recovery.cpp`) and exercise the interaction between modules:

- The kernel to daemon pipeline.
- Daemon to peer replication.
- Recovery synchronization after a node returns.

The unit and integration executables are run together with `make run_tests`.

9.4 Stress Testing

`tests/stress/stress_write.cpp` generates high-frequency writes, concurrent connections and buffer overflow conditions. Test cases TC-06 (10 concurrent writer processes) and TC-07 (10,000 chunks) exercise this category; their results are given in Section 9.6.

9.5 Memory Testing

Valgrind is used for user-space leak detection and KASAN for kernel-space checking. The goal (NFR-04) is a Valgrind-clean daemon with no memory leaks under all shutdown and error paths. Test case TC-10 additionally loads and unloads the kernel module 20 times and checks `dmesg` and `/proc/modules` for leaked resources.

9.6 Test Cases and Results

Table 19. Test cases and results

ID	Test case	Expected result	Status
TC-01	Write data to <code>/dev/vdrsd</code>	Data buffered in kernel ring, daemon woken up	Done
TC-02	Read data from <code>/dev/vdrsd</code>	Previously written data returned correctly	Done
TC-03	Write data with 3-node cluster running	Data replicated to all 3 nodes with intact CRC	Done
TC-04	Kill node2, write more data from node1	node1 and node3 store data; node2 marked OFFLINE	Done
TC-05	Restart node2 after being killed	Observer triggers <code>SYNC_REQUEST</code> ; node2 catches up missing chunks	Done
TC-06	Concurrent writes from 10 processes	All chunks stored without corruption (verified by CRC32)	Done
TC-07	Write 10,000 chunks via <code>stress_write</code>	No data loss, latency under 100 ms	Done
TC-08	Send <code>SIGTERM</code> to daemon	Graceful shutdown: write cache flushed, connections closed	Done
TC-09	Write invalid / malformed data to device	Proper error handling, no kernel panic, no crash	Done
TC-10	Load/unload kernel module 20 times	No resource leaks in <code>dmesg</code> , clean <code>/proc/modules</code>	Done

10 Execution and Demonstration

This section documents a complete run of the system: building and loading the module, starting the three-node cluster, writing data, simulating a node failure, recovery, and shutdown.

10.1 Build and Load Kernel Module

Install the toolchain, build the kernel module and the daemon with `make`, load the module, and verify that the device node exists:

Listing 14. Build and load

```
$ sudo apt install -y build-essential linux-headers-$(uname -r) g++ make
Reading package lists... Done

$ make
- Building VDRSD Linux Kernel Module -
make -C /lib/modules/6.8.0-49-generic/build M=/home/user/VDRSD/kernel modules
  CC [M]  /home/user/VDRSD/kernel/vdrsd.o
  MODPOST /home/user/VDRSD/kernel/Module.symvers
  CC [M]  /home/user/VDRSD/kernel/vdrsd.mod.o
  LD [M]  /home/user/VDRSD/kernel/vdrsd.ko
- Compiling VDRSD User-Space Daemon -
g++ -std=c++17 -Wall -Wextra -pthread -o vdrsd_daemon daemon/vdrsd_daemon.cpp

$ sudo insmod kernel/vdrsd.ko && sudo chmod 666 /dev/vdrsd
```

Figure 6 shows the kernel module loaded successfully: `lsmod` lists `vdrsd` and `/dev/vdrsd` exists as a character device (major 239).



```
abhilipsa@Ubuntu: ~/Desktop/VDRSD
x abhilipsa@Ubuntu: ~/Desktop/VDRSD — sudo ./vdrsd_daemon ./co

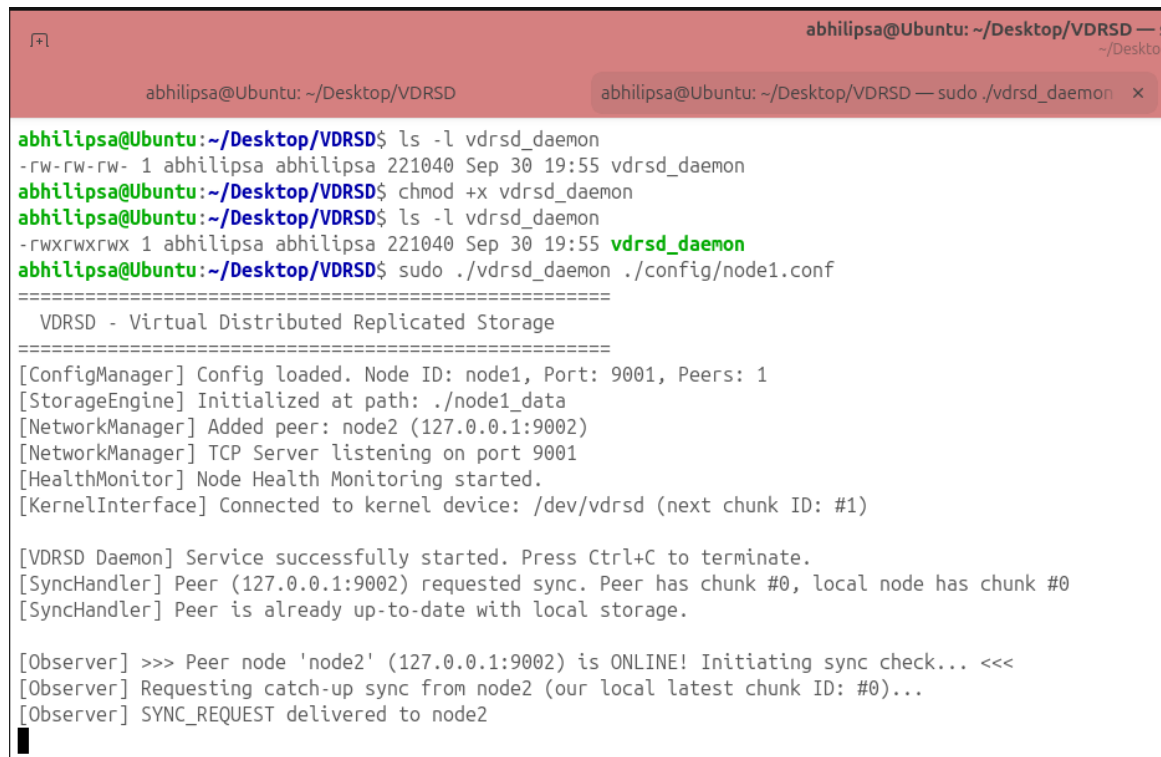
abhilipsa@Ubuntu:~/Desktop/VDRSD$ lsmod | grep vdrsd
vdrsd                16384  0
abhilipsa@Ubuntu:~/Desktop/VDRSD$ ls -l /dev/vdrsd
crw-rw-rw- 1 root root 239, 0 Oct  4 19:49 /dev/vdrsd
abhilipsa@Ubuntu:~/Desktop/VDRSD$
```

Figure 6. Kernel module loaded and `/dev/vdrsd` character device created

10.2 Start Daemon Cluster

Start the three daemons, each with its own configuration file. On start-up each daemon loads its configuration, initializes storage and networking, connects to the kernel device, starts health monitoring and registers its peers:

Figure 7 shows node1 running: its configuration is loaded, the TCP server listens on port 9001, and the kernel interface is connected to `/dev/vdrsd`.



```
abhilipsa@Ubuntu: ~/Desktop/VDRSD
abhilipsa@Ubuntu: ~/Desktop/VDRSD$ ls -l vdrsd_daemon
-rw-rw-rw- 1 abhilipsa abhilipsa 221040 Sep 30 19:55 vdrsd_daemon
abhilipsa@Ubuntu: ~/Desktop/VDRSD$ chmod +x vdrsd_daemon
abhilipsa@Ubuntu: ~/Desktop/VDRSD$ ls -l vdrsd_daemon
-rwxrwxrwx 1 abhilipsa abhilipsa 221040 Sep 30 19:55 vdrsd_daemon
abhilipsa@Ubuntu: ~/Desktop/VDRSD$ sudo ./vdrsd_daemon ./config/node1.conf
=====
VDRSD - Virtual Distributed Replicated Storage
=====
[ConfigManager] Config loaded. Node ID: node1, Port: 9001, Peers: 1
[StorageEngine] Initialized at path: ./node1_data
[NetworkManager] Added peer: node2 (127.0.0.1:9002)
[NetworkManager] TCP Server listening on port 9001
[HealthMonitor] Node Health Monitoring started.
[KernelInterface] Connected to kernel device: /dev/vdrsd (next chunk ID: #1)

[VDRSD Daemon] Service successfully started. Press Ctrl+C to terminate.
[SyncHandler] Peer (127.0.0.1:9002) requested sync. Peer has chunk #0, local node has chunk #0
[SyncHandler] Peer is already up-to-date with local storage.

[Observer] >>> Peer node 'node2' (127.0.0.1:9002) is ONLINE! Initiating sync check... <<<
[Observer] Requesting catch-up sync from node2 (our local latest chunk ID: #0)...
[Observer] SYNC_REQUEST delivered to node2
```

Figure 7. node1 daemon running and connected to the kernel device

Figure 8 shows node2 started with its own configuration file. It detects node1 as ONLINE, performs the sync check and receives replicated chunk #1.

```

abhilipsa@Ubuntu: ~/Desktop/VDRSD$ cd ~/VDRSD
./vdrsd_daemon ./config/node2.conf
bash: cd: /home/abhilipsa/VDRSD: No such file or directory
=====
VDRSD - Virtual Distributed Replicated Storage
=====
[ConfigManager] Config loaded. Node ID: node2, Port: 9002, Peers: 1
[StorageEngine] Initialized at path: ./node2_live_data
[NetworkManager] Added peer: node1 (127.0.0.1:9001)
[NetworkManager] TCP Server listening on port 9002
[HealthMonitor] Node Health Monitoring started.
[KernelInterface] Note: Character device /dev/vdrsd_node2_unused not accessible. Kernel driver bridge in fallback mode.

[VDRSD Daemon] Service successfully started. Press Ctrl+C to terminate.

[Observer] >>> Peer node 'node1' (127.0.0.1:9001) is ONLINE! Initiating sync check... <<<
[Observer] Requesting catch-up sync from node1 (our local latest chunk ID: #0)...
[Observer] SYNC_REQUEST delivered to node1
[SyncHandler] Peer (127.0.0.1:9001) requested sync. Peer has chunk #0, local node has chunk #0
[SyncHandler] Peer is already up-to-date with local storage.
[Daemon] Received replicated chunk #1 (25 bytes). Saving to local disk...

```

Figure 8. node2 daemon running, synchronizing with node1 and receiving a replicated chunk

10.3 Normal Data Write

A baseline write is made from a second terminal (Test Case 1): the text is written to /dev/vdrsd with echo and tee, the kernel module buffers it, and the daemon reads it as a chunk and flushes it to local storage.

10.4 Data Replication

After the local flush, the chunk is replicated to the peer over TCP and stored there as a chunk file.

The write was captured by the kernel, forwarded to the daemon, persisted locally, and replicated to both peers.

Figure 9 shows the replication test: data written to /dev/vdrsd on node1 appears as chunk files in the node2 storage directory.

```

abhilipsa@Ubuntu: ~/Desktop/VDRSD$ echo "TRAINER_REPLICATION_TEST" | sudo tee /dev/vdrsd
TRAINER_REPLICATION_TEST
abhilipsa@Ubuntu: ~/Desktop/VDRSD$ ls -lh node2_live_data/
total 16K
-rw-r--r-- 1 abhilipsa abhilipsa 4.1K Oct  4 19:58 chunk_00000001.dat
-rw-r--r-- 1 abhilipsa abhilipsa 4.1K Oct  4 20:02 chunk_00000002.dat
abhilipsa@Ubuntu: ~/Desktop/VDRSD$

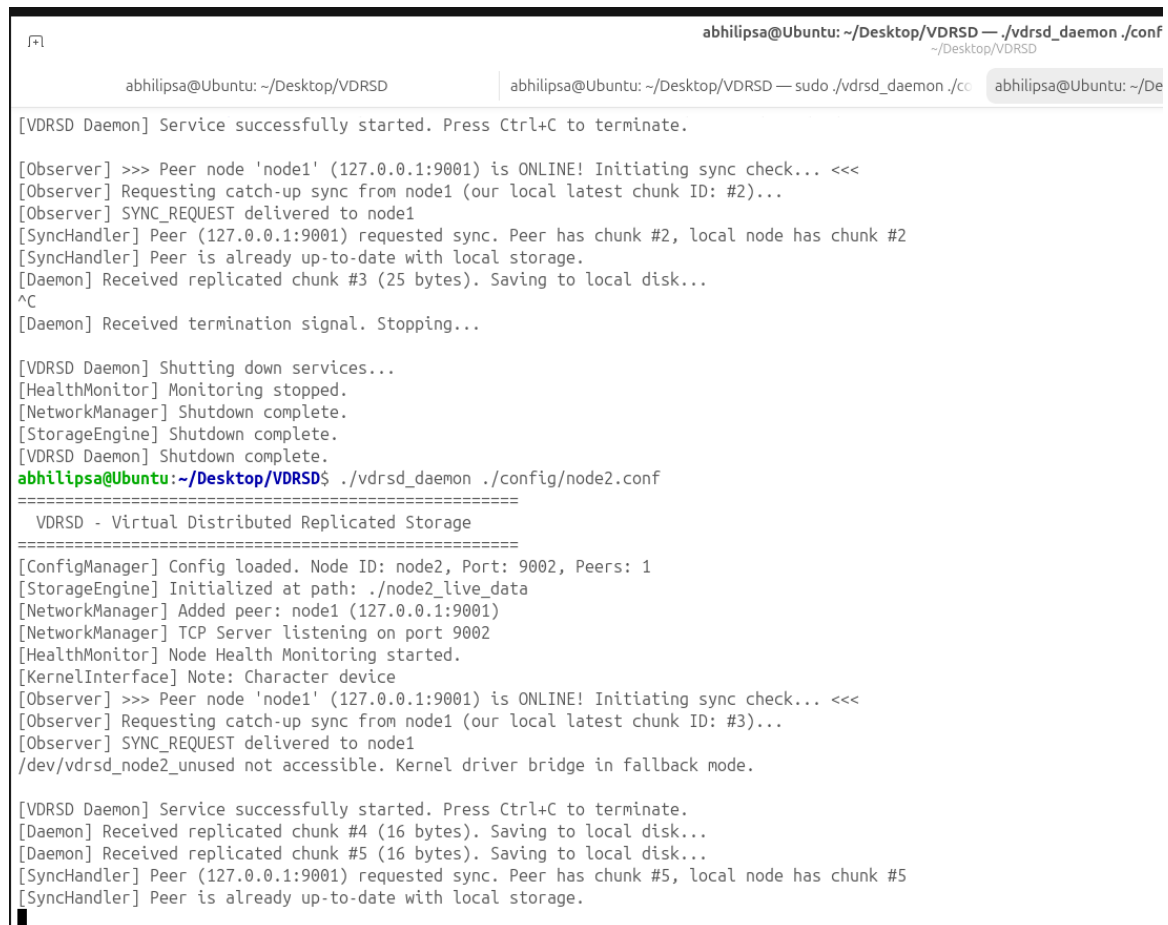
```

Figure 9. Replicated chunk files stored on node2 after a write to /dev/vdrsd

10.5 Node Failure

To test fault tolerance (Test Case 2), the node2 daemon is stopped with Ctrl+C while the cluster is running. As shown in Figure 10, it shuts down gracefully: the health monitor, network manager and storage engine stop in turn. While node2 is offline, the remaining node keeps accepting writes in degraded mode.

Figure 10 shows the node2 terminal: normal replication, then Ctrl+C, graceful shutdown and restart.



```
abhilipsa@Ubuntu: ~/Desktop/VDRSD — ./vdrsd_daemon ./conf
~/Desktop/VDRSD
abhilipsa@Ubuntu: ~/Desktop/VDRSD | abhilipsa@Ubuntu: ~/Desktop/VDRSD — sudo ./vdrsd_daemon ./cc | abhilipsa@Ubuntu: ~/De

[VDRSD Daemon] Service successfully started. Press Ctrl+C to terminate.

[Observer] >>> Peer node 'node1' (127.0.0.1:9001) is ONLINE! Initiating sync check... <<<
[Observer] Requesting catch-up sync from node1 (our local latest chunk ID: #2)...
[Observer] SYNC_REQUEST delivered to node1
[SyncHandler] Peer (127.0.0.1:9001) requested sync. Peer has chunk #2, local node has chunk #2
[SyncHandler] Peer is already up-to-date with local storage.
[Daemon] Received replicated chunk #3 (25 bytes). Saving to local disk...
^C
[Daemon] Received termination signal. Stopping...

[VDRSD Daemon] Shutting down services...
[HealthMonitor] Monitoring stopped.
[NetworkManager] Shutdown complete.
[StorageEngine] Shutdown complete.
[VDRSD Daemon] Shutdown complete.
abhilipsa@Ubuntu:~/Desktop/VDRSD$ ./vdrsd_daemon ./config/node2.conf
=====
VDRSD - Virtual Distributed Replicated Storage
=====
[ConfigManager] Config loaded. Node ID: node2, Port: 9002, Peers: 1
[StorageEngine] Initialized at path: ./node2_live_data
[NetworkManager] Added peer: node1 (127.0.0.1:9001)
[NetworkManager] TCP Server listening on port 9002
[HealthMonitor] Node Health Monitoring started.
[KernelInterface] Note: Character device
[Observer] >>> Peer node 'node1' (127.0.0.1:9001) is ONLINE! Initiating sync check... <<<
[Observer] Requesting catch-up sync from node1 (our local latest chunk ID: #3)...
[Observer] SYNC_REQUEST delivered to node1
/dev/vdrsd_node2_unused not accessible. Kernel driver bridge in fallback mode.

[VDRSD Daemon] Service successfully started. Press Ctrl+C to terminate.
[Daemon] Received replicated chunk #4 (16 bytes). Saving to local disk...
[Daemon] Received replicated chunk #5 (16 bytes). Saving to local disk...
[SyncHandler] Peer (127.0.0.1:9001) requested sync. Peer has chunk #5, local node has chunk #5
[SyncHandler] Peer is already up-to-date with local storage.
```

Figure 10. node2 daemon stopped with Ctrl+C, then restarted and synchronized with node1

The system operates in degraded mode (2 of 3 replicas) and continues to accept and store data.

10.6 Node Recovery and Synchronization

When node2 is restarted (lower part of Figure 10), it detects node1 as ONLINE, requests a catch-up sync from its latest local chunk (#3) and receives the missing chunks #4 and #5. node1 then reports that the peer is up to date. Figure 11 confirms the result: RECOVERY_DEMO_1 and RECOVERY_DEMO_2 are written to /dev/vdrsd, and chunk files #4 and #5 appear in node2_live_data.

Figure 11 shows the writes made during the demonstration and the chunk files present on node2 afterwards.

```
abhilipsa@Ubuntu: ~/Desktop/VDRSD
abhilipsa@Ubuntu: ~/Desktop/VDRSD — sudo ./vdrsd_daemon ./cc
abhilipsa@Ubuntu: ~/Desktop/VDRSD$ echo "TRAINER_REPLICATION_TEST" | sudo tee /dev/vdrsd
[sudo: authenticate] Password:
TRAINER_REPLICATION_TEST
abhilipsa@Ubuntu: ~/Desktop/VDRSD$ ls -lh node2_live_data/
total 24K
-rw-r--r-- 1 abhilipsa abhilipsa 4.1K Oct  4 19:58 chunk_00000001.dat
-rw-r--r-- 1 abhilipsa abhilipsa 4.1K Oct  4 20:02 chunk_00000002.dat
-rw-r--r-- 1 abhilipsa abhilipsa 4.1K Oct  4 20:36 chunk_00000003.dat
abhilipsa@Ubuntu: ~/Desktop/VDRSD$ echo "RECOVERY_DEMO_1" | sudo tee /dev/vdrsd
RECOVERY_DEMO_1
abhilipsa@Ubuntu: ~/Desktop/VDRSD$ echo "RECOVERY_DEMO_2" | sudo tee /dev/vdrsd
RECOVERY_DEMO_2
abhilipsa@Ubuntu: ~/Desktop/VDRSD$ ls
Makefile  README.md  chunk1  daemon  node  vdrsd  node1_data  node2_live_data  vdrsd  vdrsd_data  vdrsd_data  vdrsd  v
abhilipsa@Ubuntu: ~/Desktop/VDRSD$ ls -lh node2_live_data/
total 40K
-rw-r--r-- 1 abhilipsa abhilipsa 4.1K Oct  4 19:58 chunk_00000001.dat
-rw-r--r-- 1 abhilipsa abhilipsa 4.1K Oct  4 20:02 chunk_00000002.dat
-rw-r--r-- 1 abhilipsa abhilipsa 4.1K Oct  4 20:36 chunk_00000003.dat
-rw-r--r-- 1 abhilipsa abhilipsa 4.1K Oct  4 20:39 chunk_00000004.dat
-rw-r--r-- 1 abhilipsa abhilipsa 4.1K Oct  4 20:39 chunk_00000005.dat
abhilipsa@Ubuntu: ~/Desktop/VDRSD$
```

Figure 11. Recovery demonstration: new writes and chunk files restored on node2

Missing chunks are restored automatically, returning the cluster to full replication.

10.7 Important Terminal Outputs

The log prefixes identify the subsystem that produced each message. The most significant messages are summarized below.

Table 20. Key log messages

Message	Meaning
[KernelInterface] Read chunk #N (X bytes, CRC: ...)	The daemon read a chunk from /dev/vdrsd and computed its CRC32.
[StorageEngine] Flushed chunk #N -> ...chunk_XXXXXXX.dat	The chunk was persisted to local disk.
[NetworkManager] Replicated chunk #N -> nodeX ... ✓	The chunk was replicated to the peer successfully.
[NetworkManager] Replicated chunk #N -> nodeX ... ✗ OFFLINE	Replication failed; the peer is marked OFFLINE.
[Observer] Peer node ... went OFFLINE.	The health monitor detected a peer failure.
[Observer] Peer node ... is ONLINE! Initiating sync check...	A peer is back; the ClusterSyncObserver starts catch-up synchronization.
[NetworkManager] Catch-up sync: Sent chunk #N ...	A missed chunk was pushed to the recovering peer.
[Daemon] Received termination signal. Stopping...	Graceful shutdown has begun.

11 Results and Discussion

11.1 Achievements

Table 21. Project achievements

Achievement	Description
End-to-end pipeline	Complete path from user application to kernel, daemon, TCP network and disk storage, all working.
Custom device driver	Character device driver with proper resource management, statistics and IOCTL support.
C++17 daemon	Modern C++17 with multithreading, RAII, condition variables and zero external dependencies.
Data integrity	CRC32 checksum on every chunk, verified at receive on each replica before storage.
Fault recovery	Observer-driven catch-up sync automatically restores missed chunks when a node recovers.
Design patterns	Singleton, Observer, Producer-Consumer and RAII applied throughout the codebase.

11.2 Fault-Tolerance Behavior

When node2 was killed (TC-04), the primary node marked it OFFLINE and continued writing to its own storage and to node3, operating in degraded mode with 2 of 3 replicas. This meets the requirement that the system tolerates a single-node failure (NFR-03).

When node2 was restarted (TC-05), the `ClusterSyncObserver` sent a `SYNC_REQUEST` and the missing chunks were pushed back using `DATA_WRITE` messages, restoring full replication without manual intervention.

11.3 Data Integrity

Each `DataChunk` carries a CRC32 checksum of its payload. The checksum is verified by every replica when the chunk is received and before it is stored. In the tests, replicated data arrived with intact CRC on all three nodes (TC-03), and chunks written by 10 concurrent processes were stored without corruption, as verified by CRC32 (TC-06).

11.4 Performance and Testing Results

Table 22. Summary of measured results

Aspect	Target / expectation	Result
Replication latency	< 100 ms on localhost cluster (NFR-01)	Latency under 100 ms in the 10,000-chunk stress test (TC-07)
Data loss under load	No data loss	None in TC-07 (10,000 chunks)
Concurrent writers	No corruption	All chunks stored intact with 10 concurrent processes (TC-06)
Module load/unload	No resource leaks	Clean <code>dmesg</code> and <code>/proc/modules</code> over 20 cycles (TC-10)
Test cases completed	All planned tests	10 of 10 test cases marked Done

All results were obtained on a localhost cluster; the system has not been benchmarked across physical network interfaces (see Section 12.1).

12 Limitations and Future Work

12.1 Current Limitations

- Replication is eventually consistent, not strongly consistent (no Raft or Paxos).
- Single-writer assumption: there is no concurrent write conflict resolution.
- Localhost testing only: the system has not been benchmarked across physical network interfaces.
- No TLS or encryption on the replication channel.
- Fixed chunk size of 4 KB: variable-length chunks are not supported.
- No automatic node discovery: peers must be pre-configured.

12.2 Future Improvements

1. **Raft consensus:** implement the Raft algorithm for strong consistency guarantees.
2. **TLS encryption:** add TLS 1.3 for all replication channel traffic.
3. **Variable chunk sizes:** support streaming of large files with multi-chunk assemblies.
4. **FUSE interface:** provide an alternative user-space filesystem interface (no kernel module required).
5. **Web dashboard:** a real-time cluster health and replication statistics monitoring UI.
6. **Data deduplication:** content-addressed storage (SHA-256 based) to avoid redundant chunk storage.
7. **mDNS auto-discovery:** dynamic peer discovery on local networks.
8. **io_uring integration:** replace POSIX read/write with io_uring for dramatically lower I/O latency.

13 Module Mapping and Assessment

13.1 Training Module Coverage

Table 23. Training module coverage

Module	Topic	VDRSD implementation
M1 & M2	Computer architecture (HW/NW)	TCP/IP socket communication between distributed nodes; memory management for caching data before disk writes.
M3	Linux OS & Git	Developed entirely on Linux; Bash scripting for startup/shutdown; structured Git repository with branching strategy.
M4	C++ programming	Core daemon in C++17: OOP classes, STL (vector, mutex, atomic, thread), design patterns, template <code>RingBuffer</code> .
M5	Linux system programming	Daemon processes, POSIX file I/O, IPC (TCP sockets), signal handling, thread synchronization (mutex and condition variables).
M6	Linux device drivers	Custom character device driver exposing <code>/dev/vdrsd: file_operations, cdev, uaccess, spinlock, wait queue</code> .
M7 & M8	Software architecture & SDLC	SOLID principles, layered architecture, Singleton, Observer, Producer-Consumer; 6-stage SDLC documented.

13.2 Assessment Evidence

Table 24. Assessment evidence

Criterion	Evidence
Proper documentation	<code>README.md</code> (909 lines), <code>project_report.md</code> , the HTML documentation, inline code comments.
Git and version control	Feature branches: <code>feature/kernel-module</code> , <code>feature/daemon-core</code> , <code>feature/networking</code> , <code>feature/storage</code> .
Progress evidence	Stage-wise task completion tracking in the README with done / in-progress / planned status indicators.
Demonstration	Working terminal output, 10 documented test cases, stress test results.
Clear roadmap	"Roadmap" sections after every stage in the README.
Code quality	Static analysis (cppcheck), Valgrind memory check, SOLID principles, no raw pointers.

13.3 Git Branching Strategy

Development used a `main` / `develop` / feature-branch model. Each feature branch was merged into `develop`, and the v1.0 release was made on `main` (Figure 12).

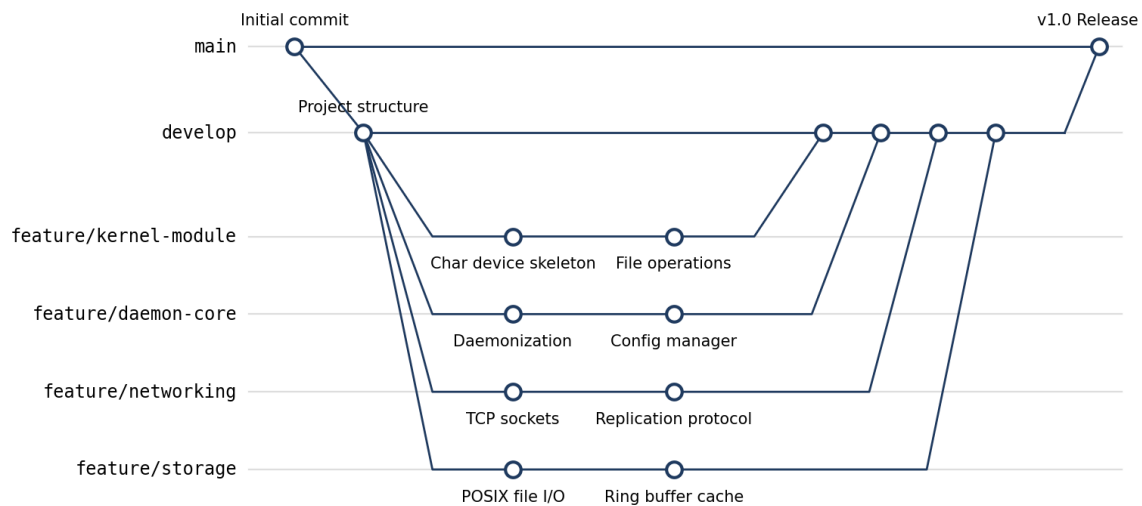


Figure 12. Git branching strategy: `main`, `develop` and feature branches

Table 25. Feature branches and their commits

Branch	Commits / topics
<code>main</code>	Initial commit; v1.0 release
<code>develop</code>	Project structure
<code>feature/kernel-module</code>	Char device skeleton; file operations
<code>feature/daemon-core</code>	Daemonization; config manager
<code>feature/networking</code>	TCP sockets; replication protocol
<code>feature/storage</code>	POSIX file I/O; ring buffer cache

13.4 Development Timeline

Figure 13 presents the six-week development plan as a Gantt chart (the weekly summary is in Section 3.3). Bar positions are read from the original project chart and show the planned order and overlap of tasks within the six stages.

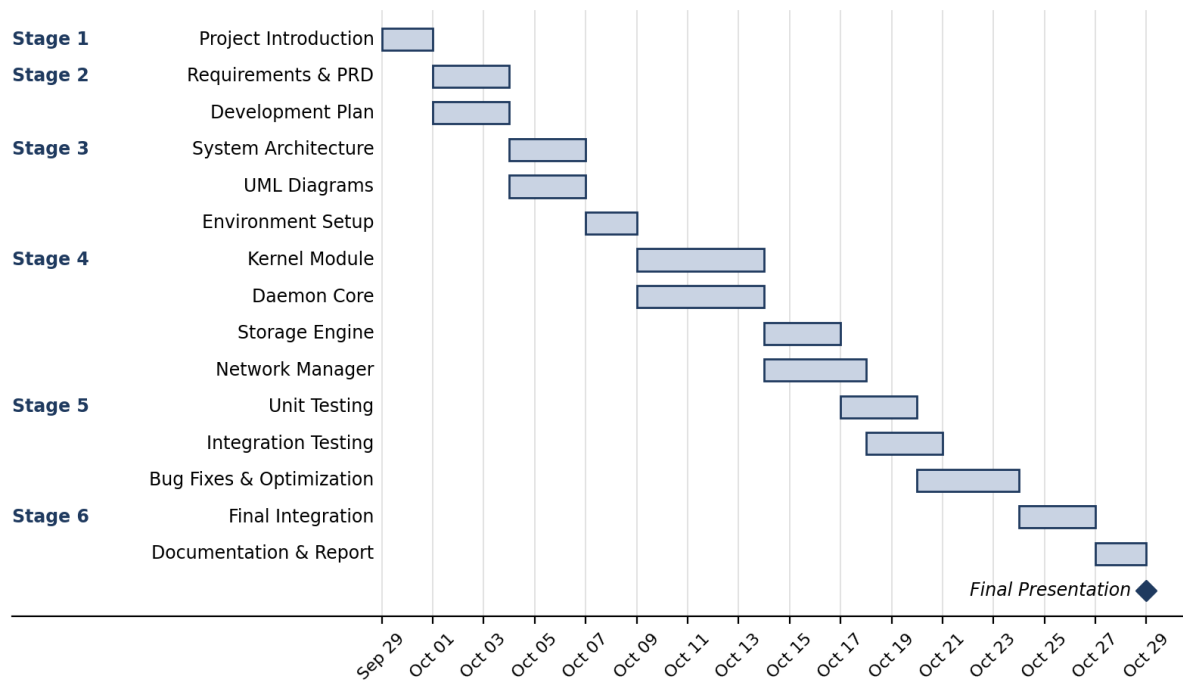


Figure 13. Project development Gantt chart (6-week timeline)

14 Conclusion

VDRSD delivers a complete end-to-end pipeline: a user application writes to `/dev/vdrsd`, a custom Linux character device driver buffers the data in a 256 KB ring buffer, and a multi-threaded C++17 daemon persists each 4 KB chunk to disk and replicates it with CRC32 verification to peer nodes over TCP.

The system tolerates the failure of a single node: it continues in degraded mode and an Observer-driven catch-up synchronization restores missed chunks when the node returns. All ten documented test cases, covering normal operation, replication, failure and recovery, concurrency, a 10,000-chunk stress run, graceful shutdown, malformed input and repeated module load/unload, were completed.

The objective was not only to complete the project, but to demonstrate a professional software development process from requirements through design, implementation and testing to final delivery. VDRSD fulfills this objective across all six stages. The limitations identified in Section 12, notably eventual consistency, localhost-only testing and the lack of encryption, define the direction of future work.

15 References and Project Resources

The following project resources are referenced in this document.

Table 26. Project resources

Resource	Description
kernel/vdrsd.c	Linux character device driver source (266 lines).
daemon/vdrsd_daemon.cpp	C++17 replication daemon source (880 lines).
README.md	Complete project documentation (909 lines).
docs/project_report.md	Project report (source documentation).
docs/vdrsd_documentation.html	HTML technical documentation.
docs/images/	Diagram sources: system_architecture.svg, class_diagram.svg, sequence_diagram.svg, state_machine.svg, git_branching_strategy.svg, project_timeline_gantt.svg.
config/ and scripts/	Node configuration files and the start, stop and status scripts.
tests/	Unit, integration and stress test sources.